Do Languages Minimise Complex Interveners?

A Cross-Linguistic Test Using Universal Dependencies

Aditya, Anya, Devansh, Nitika, Pubali, Sidhant, Srishti, and Suneet

CGS410: Language in the Mind and Machine

Abstract

Dependency Length Minimization (DLM) says that languages prefer to keep syntactically related words close together. A stronger processing-based idea is *Intervener Complexity Minimization* (ICM): when a dependency is long, the words between the head and dependent should themselves be syntactically simple. We test this idea in 12 Universal Dependencies treebanks by extracting every token that intervenes between a dependency head and dependent. Complexity is measured by whether the intervener is a syntactic head, its number of direct children, its subtree size, and its attachment relation. We compare the observed distributions with a random-word-order baseline that keeps each dependency tree fixed. Across 9,012,884 real intervener observations and 22,329,108 baseline observations, all 12 languages have significantly lower intervener head-rate, arity, and subtree size than expected under random linearisation. The results support ICM as a cross-linguistic pressure on word order.

1 Motivation for the Research Problem

Human sentence processing is limited by memory. When two syntactically related words are far apart, the listener or reader must keep one word active while processing the intervening material. DLM predicts that languages reduce this burden by shortening dependencies [1, 2]. However, distance alone may not capture the whole processing cost. A five-word gap filled with simple function words may be easier than a five-word gap filled with syntactically complex phrases.

This project asks whether natural languages reduce not only the *length* of dependencies, but also the *complexity* of the material inside long dependencies. We follow the basic idea of Yadav, Mittal, and Husain [4]: for a dependency arc from head h to dependent d, every token k whose linear position lies strictly between them is an intervener:

$$I(h, d) = \{k : \min(pos(h), pos(d)) < pos(k) < \max(pos(h), pos(d))\}.$$

Our main research question is: **Are intervening tokens in real languages syntactically simpler than they would be under random word order?** The objective is to test this question across a typologically diverse sample of 12 languages, using a single extraction and inference pipeline.

2 Hypotheses and Predictions

H1: Intervener Complexity Minimization. Real word orders should place fewer syntactic heads inside dependency spans than a random baseline that preserves the same dependency trees. The primary prediction is:

$$\Pr(\text{arity}(k) > 0 \mid k \in I)_{\text{real}} < \Pr(\text{arity}(k) > 0 \mid k \in I)_{\text{baseline}}.$$

H2: Simpler intervening structure. Real interveners should also have lower arity and smaller subtree size than baseline interveners. This predicts one-tailed Mann–Whitney U tests in the direction real $<$ baseline.

H3: Distributional difference. The full distributions of intervener part of speech, arity, and subtree size should differ from the random baseline. This predicts non-zero Jensen–Shannon divergence (JSD), especially for arity and subtree size.

3 Methods

Data. We used 12 Universal Dependencies treebanks: English, German, Spanish, Russian, Arabic, Hindi, Turkish, Finnish, Japanese, Chinese, Basque, and Ancient Greek. The sample covers different language families and word-order profiles. All train/dev/test CoNLL-U files available in each treebank were used. The final statistical comparison contains 9.01M real intervener observations and 22.33M baseline observations.

Feature extraction. For every non-adjacent dependency arc, we extracted all intervening tokens. For each intervener we recorded: (A) UPOS tag, (B) arity, defined as the number of direct UD children, (C) subtree size, defined as the number of tokens dominated by the intervener, and (D) whether the intervener attaches to the arc head, the dependent, or neither. The main ICM metric is *head-rate*: the proportion of interveners with arity > 0.

Random baseline. The null model randomly permutes word order inside each sentence while keeping the dependency tree fixed. We then re-extract interveners with the same procedure. This baseline asks what the intervener distribution would look like if the tree structure were unchanged but linear order did not minimise complexity.

Inference. We used three tests. First, head-rate was compared with a one-tailed two-proportion z-test. Second, arity and subtree-size samples were compared with one-tailed Mann–Whitney U tests, testing whether real values are stochastically smaller than baseline values. Third, JSD was used to quantify distributional distance:

$$JSD(P, Q) = \frac{1}{2}KL(P\|M) + \frac{1}{2}KL(Q\|M), \quad M = \frac{1}{2}(P + Q).$$

The significance threshold was Bonferroni-corrected over 12 languages: $\alpha = 0.05/12 = 0.0042$.

4 Results

The main prediction is supported in every language. Mean real head-rate is 0.294, while mean baseline head-rate is 0.367. The average real-baseline difference is -0.073, with all languages showing lower real head-rate than baseline. The smallest reduction is German (-0.046) and the largest is Japanese (-0.116).

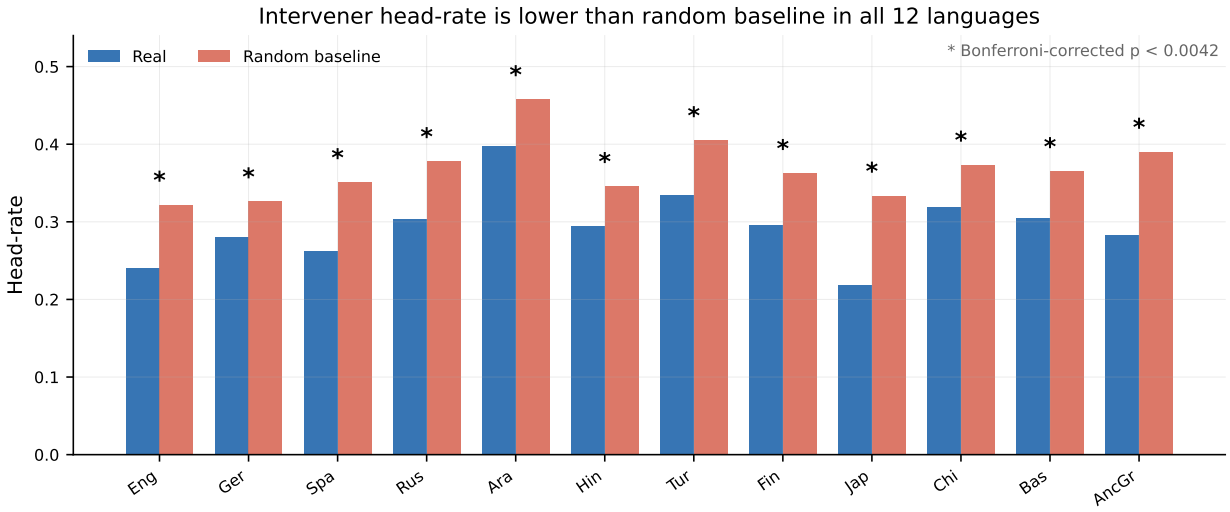
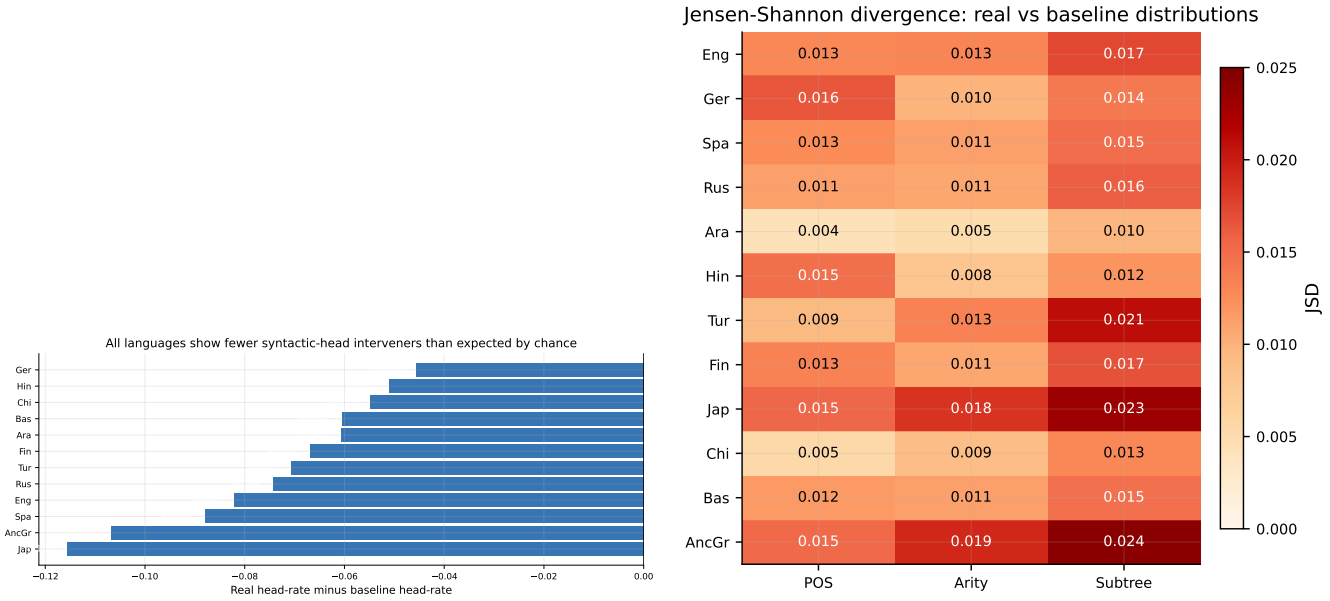


Figure 1: Primary ICM result. In all 12 languages, the proportion of interveners that are syntactic heads is lower in real word order than in the random-linearisation baseline. Stars indicate Bonferroni-corrected significance.



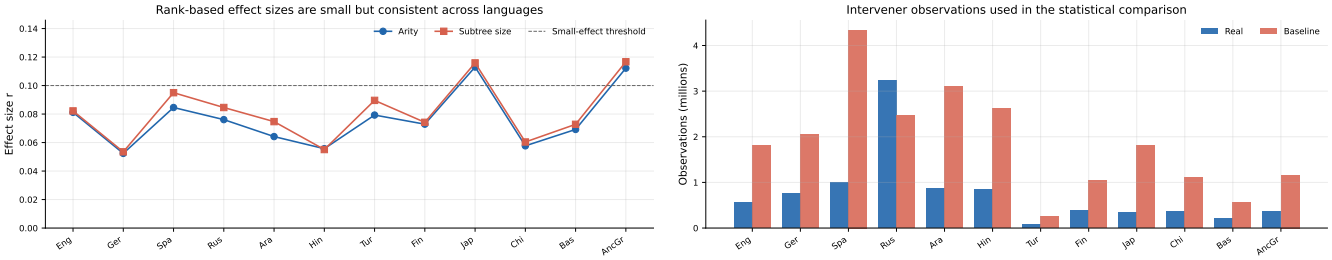
(a) Real minus baseline head-rate.

(b) JSD by feature and language.

Figure 2: All languages show a negative head-rate difference. Distributional divergence is small in absolute size, but consistent across POS, arity, and subtree size.

Table 1: Key statistical results. Head-rate tests compare real and baseline proportions. Arity and subtree p-values are from one-tailed Mann–Whitney U tests. All entries are significant at $\alpha = 0.0042$.

Language	Real head	Base head	Difference	Arity p	Subtree p
English	0.240	0.322	-0.082	1.35e-231	1.36e-236
German	0.280	0.326	-0.046	9.44e-93	7.66e-97
Spanish	0.263	0.351	-0.088	1.47e-235	4.50e-298
Russian	0.304	0.378	-0.074	3.80e-179	3.06e-220
Arabic	0.397	0.458	-0.061	7.69e-114	9.62e-152
Hindi	0.295	0.346	-0.051	1.10e-100	6.14e-99
Turkish	0.334	0.405	-0.071	3.17e-187	1.21e-235
Finnish	0.296	0.363	-0.067	2.17e-169	1.52e-174
Japanese	0.218	0.333	-0.116	0.00e+00	0.00e+00
Chinese	0.318	0.373	-0.055	3.23e-103	3.07e-113
Basque	0.304	0.365	-0.061	2.41e-151	1.94e-166
Ancient Greek	0.283	0.390	-0.107	0.00e+00	0.00e+00



(a) Rank-based effect sizes.

(b) Observation counts.

Figure 3: The statistical effects are small to modest in size, but they are extremely consistent and are estimated from large samples. Arity effect sizes range from 0.052 to 0.113; subtree effect sizes range from 0.054 to 0.117.

The JSD values are low in absolute magnitude: POS ranges from 0.0043 to 0.0165, arity from 0.0050 to 0.0193, and subtree size from 0.0097 to 0.0243. This means real and baseline distributions are not radically different overall. The important result is directional consistency: every language has lower real head-rate, lower arity, and lower subtree size than expected under the random baseline.

5 Theoretical Implications

The results support the idea that dependency locality is not only about linear distance. Languages also appear to manage the syntactic complexity of the material placed between related words. This is compatible with memory-based theories of processing: intervening syntactic heads can introduce additional incomplete dependencies and increase retrieval interference, so avoiding them should make sentence comprehension easier.

The findings also suggest a possible bridge between DLM and richer theories of word order. DLM predicts shorter dependencies; ICM predicts simpler material inside dependencies. These pressures can work together, but they are not identical. A language might allow a long dependency when the intervening material is simple, while avoiding long spans that contain complex phrase structure. This helps explain why distance-based metrics alone may miss part of the processing pressure shaping word order.

The effects are not large, so they should not be interpreted as a deterministic grammar rule. They are better understood as a statistical pressure visible across large corpora. The consistency across 12 languages, including typologically different languages, suggests that ICM is a plausible cross-linguistic tendency relevant to both human language processing and machine models of syntax.

References

[1] Gibson, E. (1998). Linguistic complexity: Locality of syntactic dependencies. *Cognition*, 68(1), 1–76.

[2] Liu, H. (2008). Dependency distance as a metric of language comprehension difficulty. *Journal of Cognitive Science*, 9(2), 159–191.

[3] Nivre, J., de Marneffe, M.-C., Ginter, F., Hajič, J., Manning, C. D., Pyysalo, S., Schuster, S., Tyers, F., and Zeman, D. (2020). Universal Dependencies v2: An evergrowing multilingual treebank collection. *Proceedings of LREC*.

[4] Yadav, H., Mittal, S., and Husain, S. (2022). A reappraisal of dependency length minimization as a linguistic universal. *Open Mind*, 6, 147–168. https://doi.org/10.1162/opmi_a_00060.

A Implementation Details

Pipeline. The implementation is organised as a reproducible sequence of scripts:

1. `step2_verify_data.py`: validate CoNLL-U files.
2. `step3_extract_interveners.py`: extract one row per arc-intervener pair.
3. `step4_compute_features.py`: aggregate POS, arity, subtree, attachment, and head-rate statistics.
4. `step5_baseline_generator.py`: build the random-linearisation baseline.
5. `step6_statistical_tests.py`: run JSD, Mann–Whitney U, and head-rate tests.
6. `step7_visualizations.py`: generate the full figure set.

Core extraction code. The following excerpt implements the formal definition of an intervener and records the four project features.

```
--→-→-←←←
def extract_interveners_from_sentence(
    sentence: Sentence,
    language: str
) -> List[Dict]:
    """
    Extract all intervener observations from a single sentence.

    For each dependency arc (h → d) where |pos(h) - pos(d)| > 1:
        Find all tokens k where min(h, d) < k < max(h, d)
        Record features A, B, C, D for each intervener k

    Args:
        sentence: A parsed Sentence object
        language: Language name (for output rows)

    Returns:
        List of dicts, one per (arc, intervener) pair. If a sentence has
        3 arcs each with 2 interveners, this returns 6 rows.

    KEY FORMULA (from Yadav et al. 2022, extended in our proposal):
        For arc head h at position i, dependent d at position j:
            interveners = {k : min(i,j) < k < max(i,j)}
    """
    rows = []
    tokens = sentence.tokens
    n = len(tokens)

    # Build a fast lookup: token_idx → token object
    # (tokens are already 1-indexed and contiguous after our parser)
    tok_map = {t.idx: t for t in tokens}

    typo = TYPOLOGY.get(language, {})

    for dep_tok in tokens:
        # Skip root token (head = 0, no arc to extract)
        if dep_tok.head <= 0:
            continue

        head_idx = dep_tok.head
        dep_idx = dep_tok.idx

        # Skip self-loops (should not occur in valid UD, but defensive check)
        if head_idx == dep_idx:
            continue

        arc_distance = abs(head_idx - dep_idx)

        # Only process non-adjacent arcs (adjacent arcs have no interveners)
```

```
        if arc_distance <= 1:
            continue

        # Determine the intervening span
        left = min(head_idx, dep_idx)
        right = max(head_idx, dep_idx)
        # Interveners are strictly between left and right (exclusive)
        intervener_indices = list(range(left + 1, right))

        if not intervener_indices:
            continue  # defensive; shouldn't happen when arc_distance > 1

        head_tok = tok_map.get(head_idx)
        if head_tok is None:
            continue  # malformed sentence

        num_interveners = len(intervener_indices)

        for k_idx in intervener_indices:
            k_tok = tok_map.get(k_idx)
            if k_tok is None:
                continue  # gap in token sequence (shouldn't happen)

            # --- Feature A: UPOS ---
            intervener_upos = k_tok.upos

            # --- Feature B: Arity ---
            intervener_arity = sentence.arity(k_idx)

            # --- Feature C: Subtree size ---
            intervener_subtree = sentence.subtree_size(k_idx)

            # --- Feature D: Attachment type ---
            attachment = sentence.attachment_type(k_idx, head_idx, dep_idx)

            # Whether this intervener is itself a head (arity > 0)
            # This is the key measure in Yadav et al. (2022):
            # "Intervener Complexity = number of intervening HEADS"
            is_head = 1 if intervener_arity > 0 else 0

            row = {
                "language":                language,
                "word_order":              typo.get("word_order", "unknown"),
                "morph_type":              typo.get("morph_type", "unknown"),
                "family":                  typo.get("family", "unknown"),
                "sent_id":                 sentence.sent_id or "",
                "sent_length":             n,
                "head_idx":                head_idx,
                "dep_idx":                 dep_idx,
                "head_upos":               head_tok.upos,
                "dep_upos":                dep_tok.upos,
                "head_deprel":             head_tok.deprel,
                "dep_deprel":              dep_tok.deprel,
                "arc_distance":            arc_distance,
                "num_interveners":         num_interveners,
                "intervener_idx":          k_idx,
                "intervener_form":         k_tok.form,
                "intervener_upos":         intervener_upos,
                "intervener_arity":        intervener_arity,
                "intervener_subtree_size": intervener_subtree,
                "attachment_type":         attachment,
                "is_head":                 is_head,
            }
            rows.append(row)

    return rows
```

Inference code. The statistical script implements the following logic; the full runnable version is in `step6_statistical_tests.py`.

```
def jensen_shannon_divergence(p, q):
```

```python
    m = [0.5 * (pi + qi) for pi, qi in zip(p, q)]
    jsd = 0.5 * kl_divergence(p, m) + 0.5 * kl_divergence(q, m)
    return jsd / math.log(2)  # convert to bits

def mann_whitney_test(real_sample, baseline_sample):
    # One-tailed test: real values are lower than baseline values.
    u_stat, p_value = stats.mannwhitneyu(
        real_sample, baseline_sample, alternative="less"
    )
    n1, n2 = len(real_sample), len(baseline_sample)
    mean_u = n1 * n2 / 2
    std_u = math.sqrt(n1 * n2 * (n1 + n2 + 1) / 12)
    z_score = (u_stat - mean_u) / std_u
    effect_r = abs(z_score) / math.sqrt(n1 + n2)
    return p_value, effect_r
```

B Complete Figure Appendix

The main report uses the most compact result figures. The remaining generated figures from `data/figures` are included here for completeness.

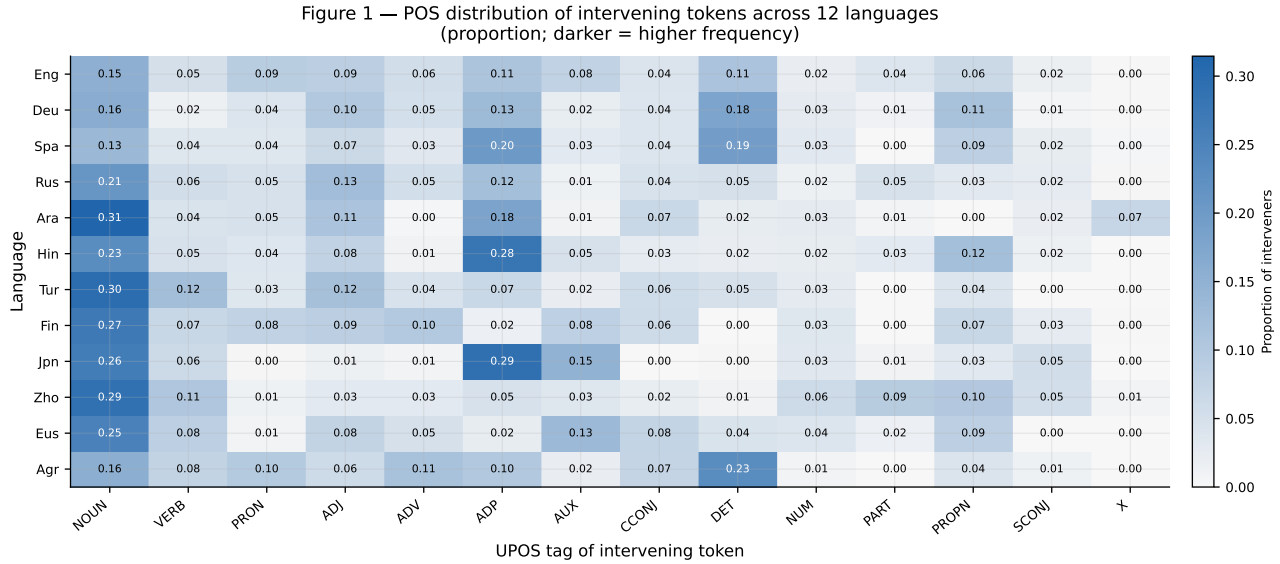


Figure 4: Original project Figure 1: POS distribution heatmap.

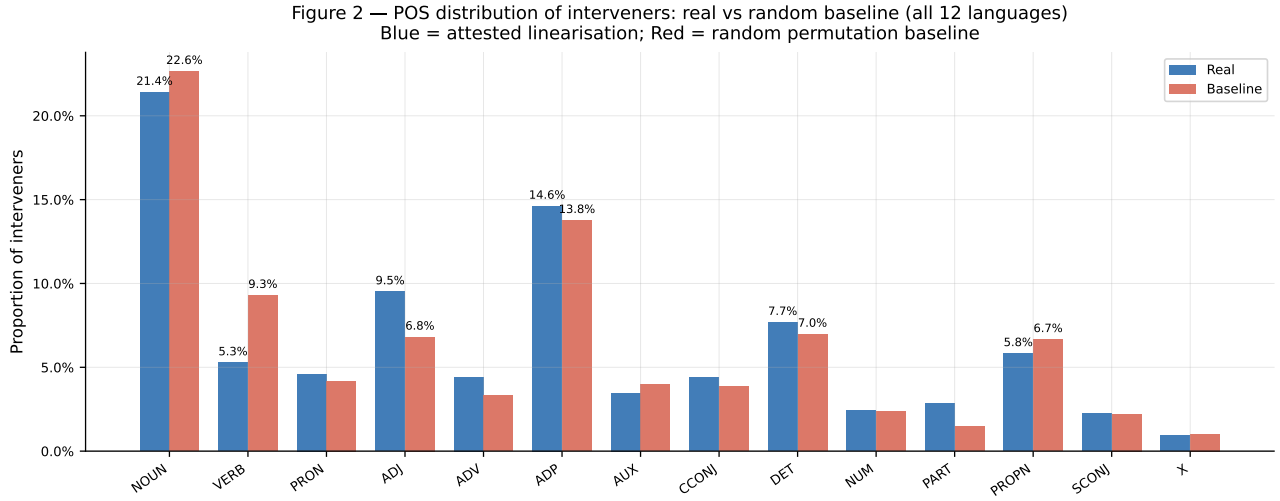


Figure 5: Original project Figure 2: POS distribution, real versus baseline.

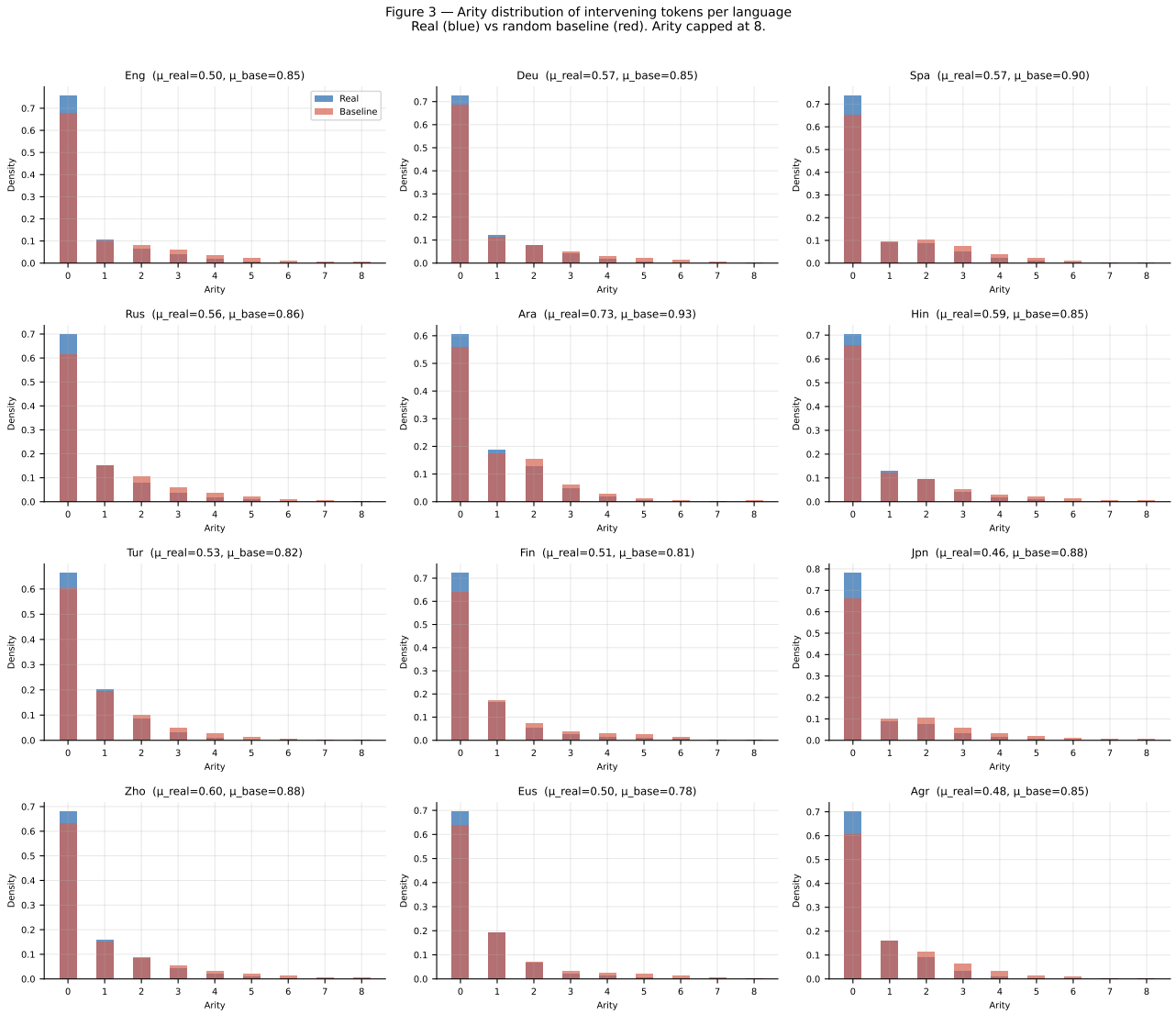


Figure 6: Original project Figure 3: arity distributions by language.

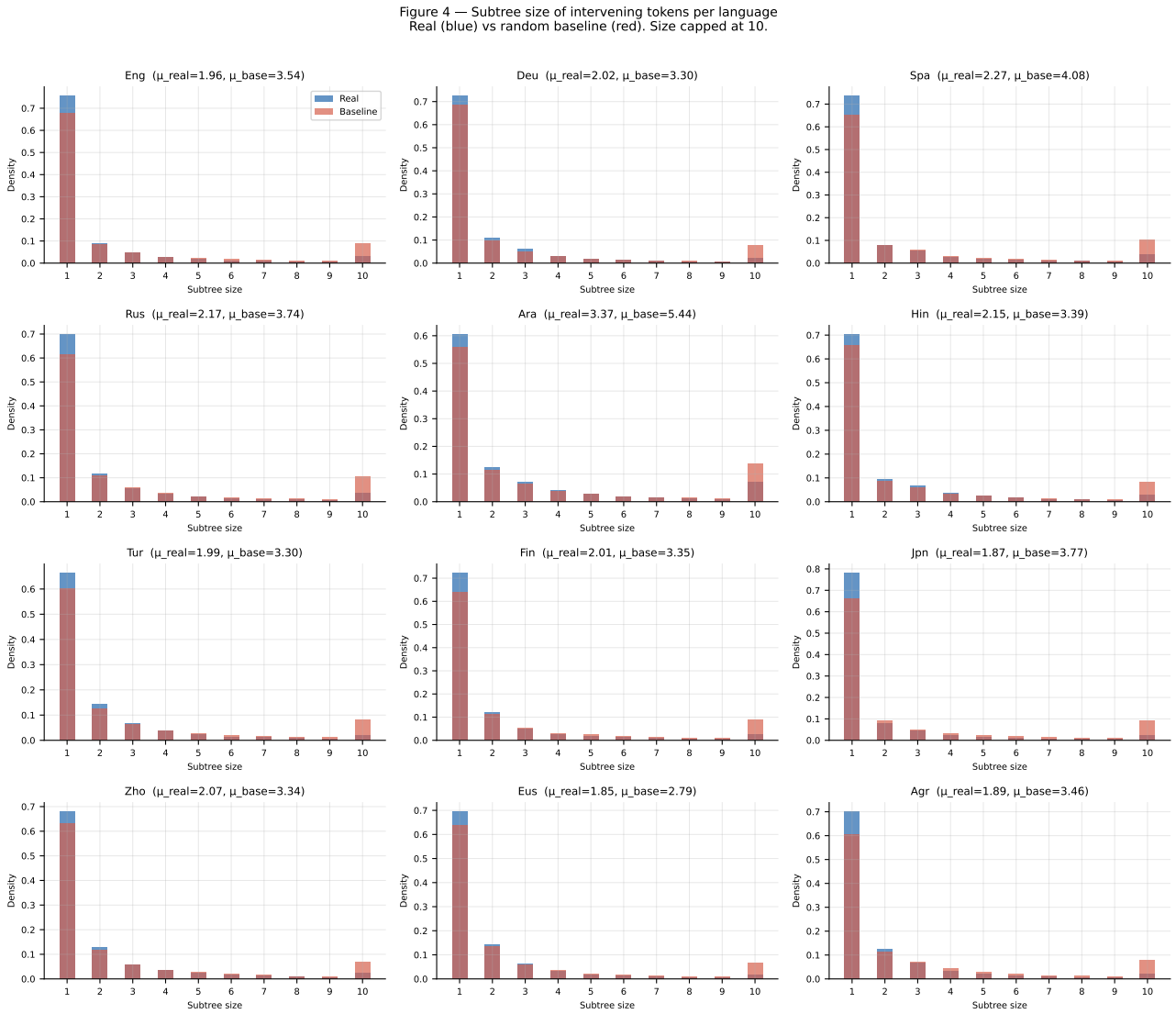


Figure 7: Original project Figure 4: subtree-size distributions by language.

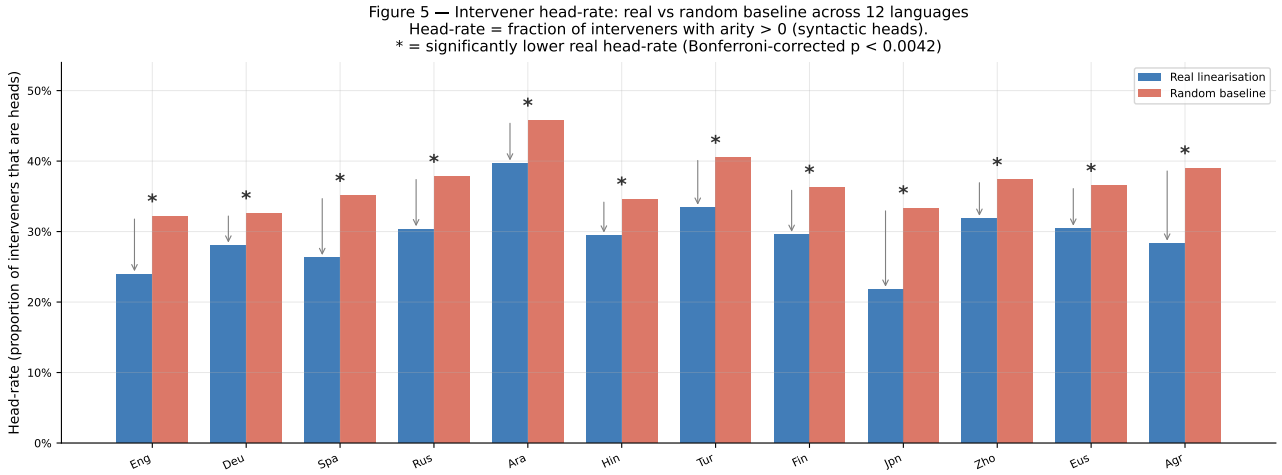


Figure 8: Original project Figure 5: head-rate comparison.

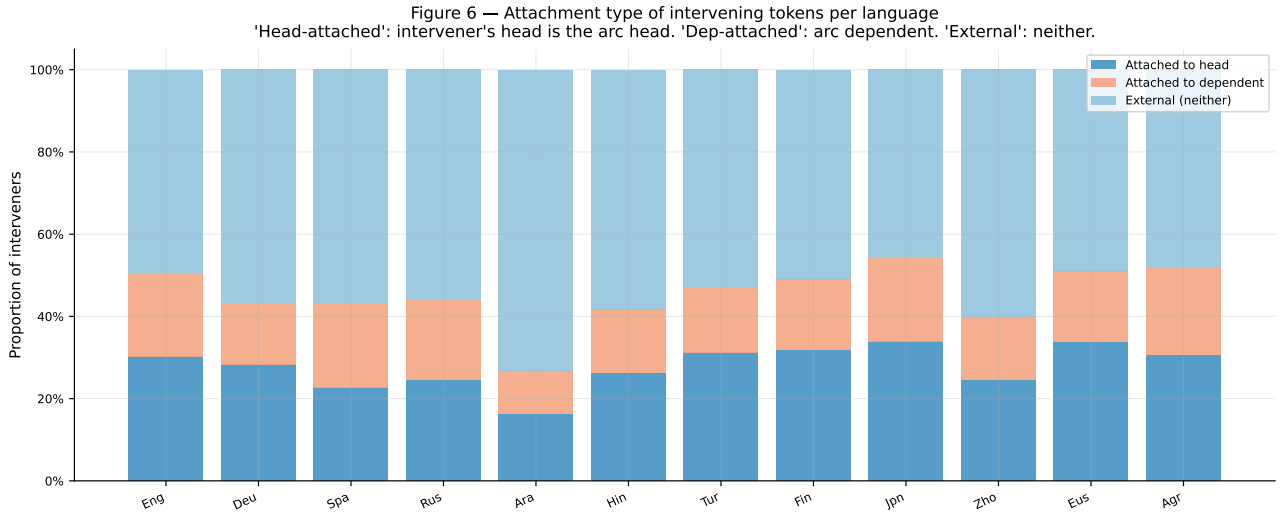


Figure 9: Original project Figure 6: attachment type distribution.

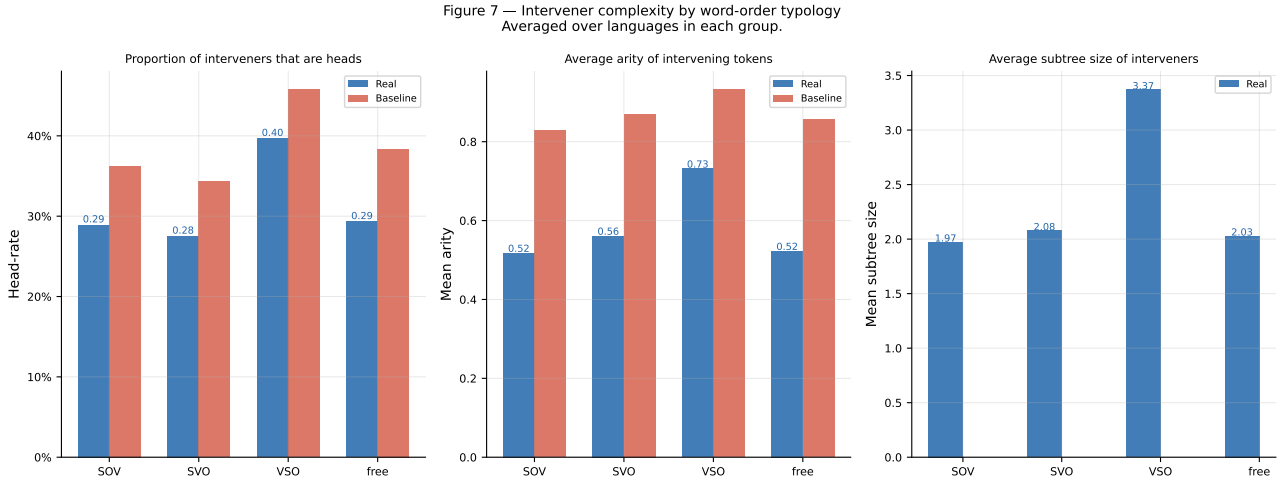


Figure 10: Original project Figure 7: word-order group comparison.

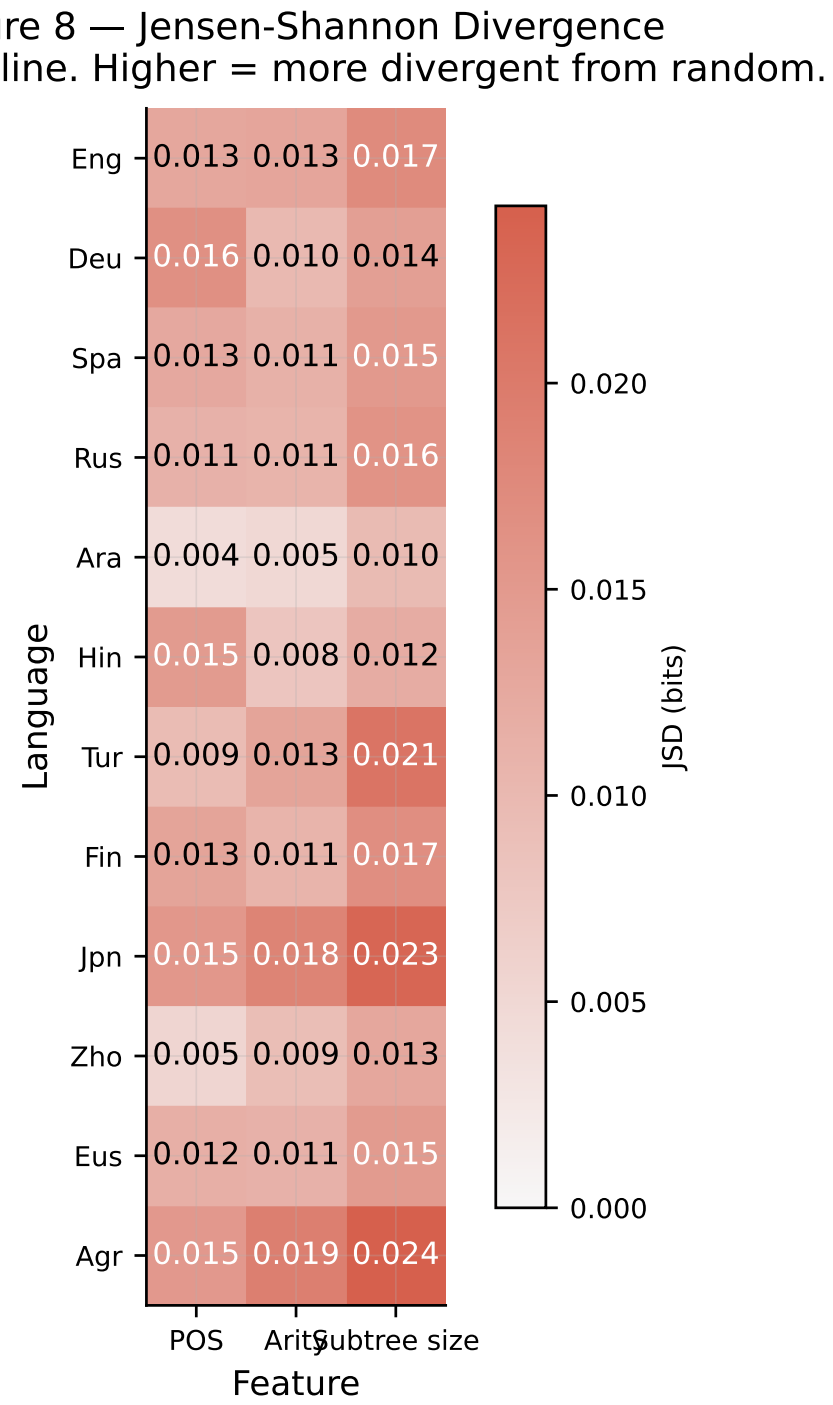


Figure 11: Original project Figure 8: JSD heatmap.

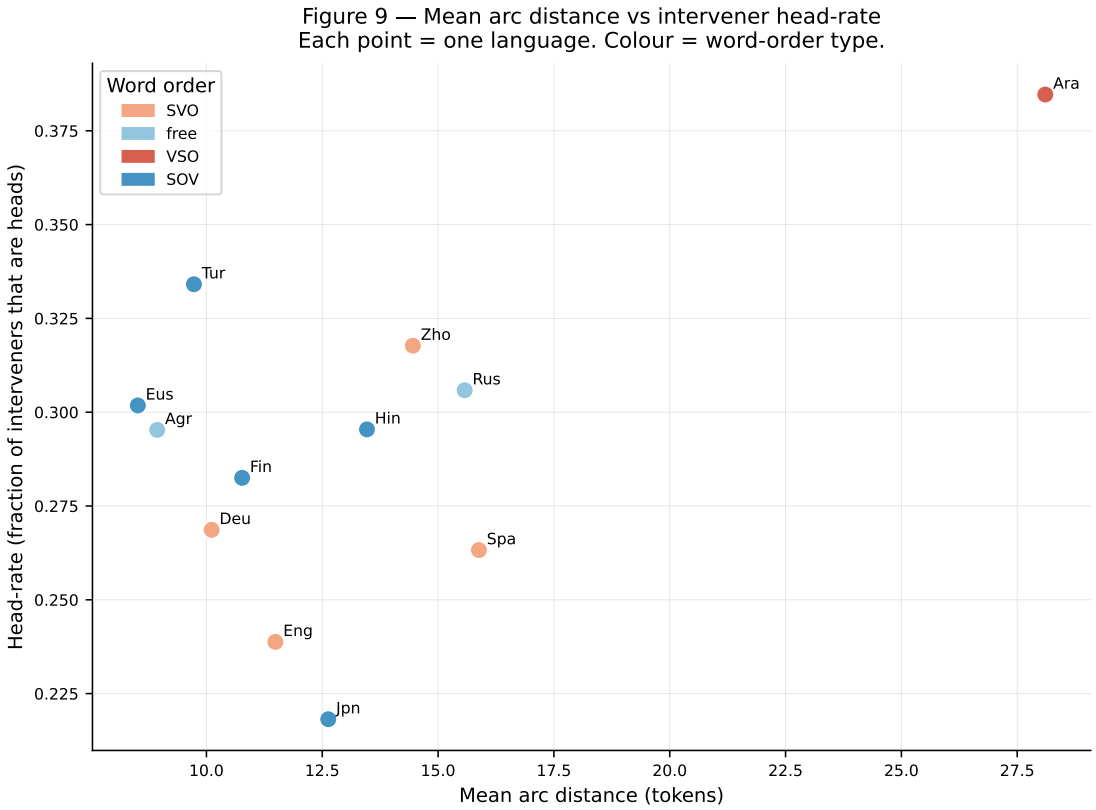


Figure 12: Original project Figure 9: mean arc distance versus intervener head-rate.

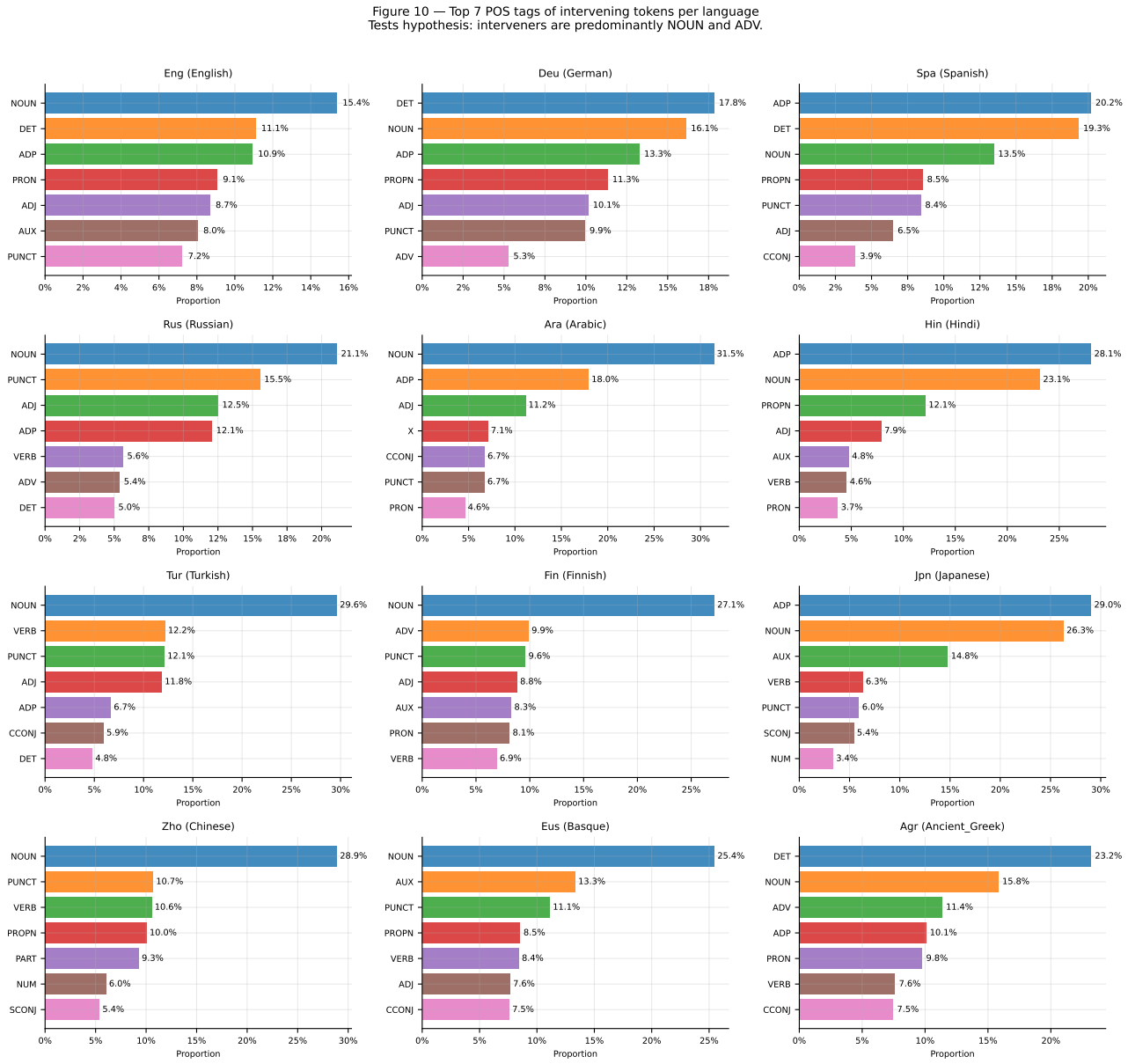


Figure 13: Original project Figure 10: top POS tags of interveners by language.